

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

EXPERIMENTS

Name of the Student	Aryan.ks
Reg. No	192325051

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I Aryan.KS / 192325051 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student: Aryan.KS

A handwritten signature in black ink, appearing to read 'Aryan', is written on a light gray rectangular background.

QUESTIONS:10

Total Marks:50

CO1- AT3 Experiments
Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Learning Algorithms – Linear Regression using Gradient Descent

Linear Regression is a supervised machine learning algorithm used to predict continuous values by finding the best-fit straight line between the input variable (x) and the output variable (y). The mathematical model of Linear Regression is given by:

$$\hat{y} = mx + c$$

where $\hat{y}$ is the predicted value, m is the slope (weight), c is the intercept (bias), and x is the input feature. The objective of the algorithm is to determine the optimal values of m and c so that the prediction error is minimized.

To measure the prediction error, the Mean Squared Error (MSE) cost function is used:

$$J(m, c) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where n is the total number of training samples, y_i is the actual value, and $\hat{y}_i$ is the predicted value. A lower cost indicates a better-fitting regression line.

The parameters are optimized using Gradient Descent, an iterative optimization algorithm. It updates the slope and intercept in the direction that reduces the cost function using the equations:

$$m = m - \alpha \frac{\partial J}{\partial m}$$
$$c = c - \alpha \frac{\partial J}{\partial c}$$

where α is the learning rate. During each iteration, the model calculates predictions, computes the error, determines the gradients, and updates the parameters. This process continues until the cost function reaches its minimum value.

For implementation, a sample dataset is trained with a learning rate of 0.01 for 1000 iterations. Initially, the loss is high because the model parameters are randomly initialized. As Gradient

Descent updates the parameters, the loss decreases steadily, indicating that the model is learning the relationship between the input and output variables.

The reduction in loss is visualized using a learning curve, where the x-axis represents the number of iterations and the y-axis represents the Mean Squared Error. The curve decreases rapidly in the beginning and gradually becomes stable, showing that the model has successfully converged to the optimal solution.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the unknown parameters of a probability distribution from observed data. In this experiment, synthetic data is generated from a Normal (Gaussian) Distribution with known mean (μ) and variance (σ^2). The goal of MLE is to estimate these parameters by finding the values that maximize the likelihood of observing the given data. The probability density function (PDF) of a normal distribution is:

$$f(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

For a dataset containing n independent observations, the likelihood function is:

$$L(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

To simplify the calculations, the log-likelihood function is used:

$$\log L = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

By maximizing the log-likelihood function, the Maximum Likelihood Estimates of the parameters are obtained as:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$
$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

In this experiment, synthetic data is generated using known parameters, for example Mean = 50 and Variance = 25 (Standard Deviation = 5). The MLE algorithm is then applied to estimate these parameters from the generated data. Since the data is randomly generated, the estimated values may not exactly match the actual parameters but will be very close, especially when the

dataset contains a large number of samples. Any small difference is due to random sampling variation.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Answer : Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate the unknown parameters of a probability distribution from observed data. In this experiment, synthetic data is generated from a Normal (Gaussian) Distribution with known mean (μ) and variance (σ^2). The goal of MLE is to estimate these parameters by finding the values that maximize the likelihood of observing the given data. The probability density function (PDF) of a normal distribution is:

$$f(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

For a dataset containing n independent observations, the likelihood function is:

$$L(\mu, \sigma^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right)$$

To simplify the calculations, the log-likelihood function is used:

$$\log L = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

By maximizing the log-likelihood function, the Maximum Likelihood Estimates of the parameters are obtained as:

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$
$$\hat{\sigma}^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

In this experiment, synthetic data is generated using known parameters, for example Mean = 50 and Variance = 25 (Standard Deviation = 5). The MLE algorithm is then applied to estimate these parameters from the generated data. Since the data is randomly generated, the estimated values may not exactly match the actual parameters but will be very close, especially when the dataset contains a large number of samples. Any small difference is due to random sampling variation.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

ANSWER:

A **Neural Network** is a machine learning model inspired by the human brain. It consists of an **input layer**, one or more **hidden layers**, and an **output layer**. In this experiment, a simple neural network with **one hidden layer** is designed and trained on a small dataset to learn **non-linear patterns**. Unlike Linear Regression, which can only model linear relationships, a neural network can capture complex non-linear relationships using activation functions.

The output of each neuron is computed as:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- x_i = Input feature
- w_i = Weight
- b = Bias
- z = Weighted sum

The hidden layer applies an activation function such as the **Sigmoid** function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For the output layer, the predicted value is:

$$\hat{y} = f(z)$$

where $f(z)$ is the activation function used in the output layer.

The network is trained using **Forward Propagation** and **Backpropagation**. During forward propagation, the inputs pass through the hidden layer to produce predictions. The prediction error is calculated using the **Mean Squared Error (MSE)** loss function:

$$Loss = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The error is then propagated backward through the network to update the weights using Gradient Descent. This process is repeated for several epochs until the loss becomes very small and the network learns the underlying pattern.

In this experiment, a small dataset is used to train the network. Initially, the predictions are inaccurate because the weights are randomly initialized. As the training progresses, the weights are updated continuously, reducing the loss and improving prediction accuracy. The hidden layer enables the model to learn **non-linear relationships**, which cannot be represented by a simple linear equation.

Result:

A simple neural network with one hidden layer was successfully trained on a small dataset. The network learned the non-linear patterns by updating its weights through backpropagation, and the prediction error decreased over successive training iterations, resulting in improved model accuracy

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :**Implement a Single Artificial Neuron using Different Activation Functions (Sigmoid and ReLU)**

An Artificial Neuron is the basic building block of an Artificial Neural Network (ANN). It receives one or more input values, multiplies them by corresponding weights, adds a bias, and passes the result through an activation function to produce the output. The mathematical model of a neuron is given by:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- x_i = Input values
- w_i = Weights
- b = Bias
- z = Weighted sum

The weighted sum is then passed through an activation function. Two commonly used activation functions are Sigmoid and ReLU.

The Sigmoid Activation Function maps the input to a value between 0 and 1, making it suitable for binary classification problems. It is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The ReLU (Rectified Linear Unit) activation function outputs the input directly if it is positive; otherwise, it returns zero. It is defined as:

$$ReLU(z) = \max(0, z)$$

In this experiment, a single artificial neuron is implemented using the same input values, weights, and bias while applying both Sigmoid and ReLU activation functions. The weighted sum is first calculated, and then each activation function produces its respective output. The outputs are compared to understand their behavior. The Sigmoid function produces values between 0 and 1, making the output smooth and suitable for probability estimation. In contrast, the ReLU function outputs 0 for negative inputs and the original value for positive inputs, making it computationally efficient and widely used in deep neural networks.

For example, if the weighted sum is:

$$z = 2.5$$

then,

$$\sigma(2.5) = \frac{1}{1 + e^{-2.5}} \approx 0.924$$
$$ReLU(2.5) = 2.5$$

Comparison:

Activation Function Output for $z = 2.5$

Sigmoid	0.924
ReLU	2.5

Result:

A single artificial neuron was successfully implemented using Sigmoid and ReLU activation functions. For the same input values, Sigmoid produced an output between 0 and 1, while ReLU returned the positive input directly. The comparison shows that ReLU is computationally simpler and faster, whereas Sigmoid is useful when the output needs to represent a probability

6. Perceptron and Multilayer Perceptron (MLP)

- Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

The Perceptron is one of the simplest artificial neural network models used for binary classification. It classifies input data into one of two classes by learning a linear decision boundary. The perceptron receives input features, multiplies them by corresponding weights, adds a bias, and passes the result through an activation function to produce the output. It works effectively only when the data is linearly separable, meaning the two classes can be separated using a single straight line (or hyperplane).

The weighted sum of the inputs is calculated as:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

x_i = Input features

w_i = Weights

b = Bias

z = Weighted sum

The perceptron uses a step activation function to determine the output:

$$y = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

During training, the weights are updated whenever the prediction is incorrect using the perceptron learning rule:

$$w_{\text{new}} = w_{\text{old}} + \eta(y - \hat{y})x$$

where:

η = Learning rate

y = Actual output

$\hat{y}$ = Predicted output

In this experiment, the Perceptron algorithm is trained on two datasets: a linearly separable dataset (such as the AND gate) and a non-linearly separable dataset (such as the XOR gate). For the linearly separable dataset, the perceptron successfully learns the correct decision boundary after several training iterations and achieves high classification accuracy. However, when trained on the XOR dataset, the perceptron fails to classify the data correctly because a single straight line cannot separate the two classes. As a result, the algorithm does not converge and produces incorrect predictions.

- b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Answer :

Train a Multilayer Perceptron (MLP) Model and Compare its Performance with a Single-Layer Perceptron

A Multilayer Perceptron (MLP) is an advanced type of Artificial Neural Network that consists of an input layer, one or more hidden layers, and an output layer. Unlike a single-layer perceptron, an MLP can learn both linear and non-linear relationships by using hidden layers and non-linear activation functions such as ReLU, Sigmoid, or Tanh. The hidden layers enable the network to extract complex features from the input data, making it suitable for solving more challenging classification problems.

The output of each neuron is calculated as:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- x_i = Input features
- w_i = Weights
- b = Bias
- z = Weighted sum

The hidden layer applies an activation function such as the Sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The network is trained using Forward Propagation and Backpropagation. During forward propagation, the input passes through the hidden layer to produce predictions. The prediction error is calculated using the Mean Squared Error (MSE) loss function:

$$Loss = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The error is then propagated backward, and the weights are updated using Gradient Descent. This process is repeated for several epochs until the loss is minimized and the network achieves good prediction accuracy.

In this experiment, the MLP is trained on the same dataset used for the single-layer perceptron. The single-layer perceptron performs well only for linearly separable data, whereas the MLP successfully learns both linear and non-linear patterns due to the presence of hidden layers. As training progresses, the MLP reduces the prediction error significantly and achieves higher classification accuracy

7. Forward Propagation and Backpropagation Algorithm

- a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Forward Propagation and Backpropagation Algorithm

Manually Compute Forward Propagation for a Small Neural Network and Verify the Output Using Code

Forward Propagation is the process by which input data passes through a neural network to produce the final output. Each neuron calculates a weighted sum of its inputs, adds a bias, and applies an activation function. During training, the output is compared with the actual value, and the error is minimized using the Backpropagation algorithm, which updates the weights to improve the network's performance.

For a neuron, the weighted sum is calculated as:

$$z = \sum_{i=1}^n w_i x_i + b$$

where:

- x_i = Input values
- w_i = Weights
- b = Bias
- z = Weighted sum

The output of the neuron is obtained using the Sigmoid activation function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Consider a small neural network with two inputs:

- $x_1 = 0.5$
- $x_2 = 0.8$

Weights:

- $w_1 = 0.4$
- $w_2 = 0.7$
- Bias:
- $b = 0.2$

The weighted sum is calculated as:

$$z = (0.5 \times 0.4) + (0.8 \times 0.7) + 0.2$$

$$z = 0.20 + 0.56 + 0.20 = 0.96$$

Applying the Sigmoid activation function:

$$\hat{y} = \frac{1}{1 + e^{-0.96}}$$

$$\hat{y} \approx 0.723$$

Thus, the manually computed output of the neuron is 0.723.

During Backpropagation, the prediction error is calculated using the Mean Squared Error (MSE):

$$Loss = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

The error is propagated backward through the network, and the weights are updated using Gradient Descent:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial Loss}{\partial w}$$

where:

- η = Learning rate
- $\frac{\partial Loss}{\partial w}$ = Gradient of the loss with respect to the weight

The process of forward propagation and backpropagation is repeated until the network minimizes the error and achieves accurate predictions.

Verification:

The manually calculated output is verified using Python code by implementing the same inputs, weights, bias, and Sigmoid activation function. The code produces an output of approximately 0.723, confirming that the manual calculations are correct.

Result:

Forward propagation was successfully computed manually for a small neural network and verified using code. The manually calculated output matched the program output, demonstrating the correctness of the forward propagation process. Backpropagation updates the network weights based on the prediction error, enabling the neural network to learn and improve its accuracy over successive training iterations

- b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Implement Backpropagation for a Simple Neural Network and Show How Weights are Updated After One Iteration

Backpropagation is a supervised learning algorithm used to train artificial neural networks. It minimizes the prediction error by propagating the error backward through the

network and updating the weights using Gradient Descent. The objective is to reduce the loss function so that the network produces more accurate predictions after each training iteration.

The output of a neuron is computed using the weighted sum:

$$z = \sum_{i=1}^n w_i x_i + b$$

The output is then passed through the Sigmoid activation function:

$$\hat{y} = \frac{1}{1 + e^{-z}}$$

The prediction error is measured using the Mean Squared Error (MSE):

$$Loss = \frac{1}{2} (y - \hat{y})^2$$

where:

- y = Actual output
- $\hat{y}$ = Predicted output

The weights are updated using the Gradient Descent rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial Loss}{\partial w}$$

where:

- η = Learning rate
- $\frac{\partial Loss}{\partial w}$ = Gradient of the loss with respect to the weight

Consider a simple neural network with:

- Input $x = 1.0$
- Initial weight $w = 0.5$
- Bias $b = 0.2$
- Learning rate $\eta = 0.1$
- Target output $y = 1$

Step 1: Forward Propagation

$$z = (1.0 \times 0.5) + 0.2 = 0.7$$
$$\hat{y} = \frac{1}{1 + e^{-0.7}} \approx 0.668$$

Step 2: Compute Loss

$$Loss = \frac{1}{2} (1 - 0.668)^2 \approx 0.055$$

Step 3: Backpropagation

The gradient of the loss is computed, and the weight is updated:

$$\begin{aligned}w_{\text{new}} &= 0.5 - 0.1(-0.073) \\w_{\text{new}} &\approx 0.507\end{aligned}$$

Thus, after one iteration, the weight changes from 0.500 to 0.507, reducing the prediction error

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Implement Gradient Descent to Minimize a Cost Function and Analyze the Effect of Learning Rate on Convergence Speed

Gradient Descent is an optimization algorithm used to minimize the cost function of a machine learning model by iteratively updating its parameters. The objective is to find the optimal values of weights and bias that produce the minimum prediction error. It is widely used in algorithms such as Linear Regression, Logistic Regression, and Neural Networks. Gradient Descent works by moving in the direction opposite to the gradient of the cost function, thereby reducing the loss at each iteration.

The cost function commonly used for regression problems is the Mean Squared Error (MSE):

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- $J(\theta)$ = Cost function
- n = Number of training samples
- y_i = Actual output
- $\hat{y}_i$ = Predicted output

The parameters are updated using the Gradient Descent update rule:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial J}{\partial \theta}$$

where:

- θ = Model parameter (weight or bias)
- α = Learning rate
- $\frac{\partial J}{\partial \theta}$ = Gradient of the cost function

In this experiment, Gradient Descent is implemented to minimize the cost function of a simple regression model. Initially, the cost is high because the model parameters are randomly initialized. During each iteration, the gradients are computed, and the parameters are updated. As training continues, the cost function decreases steadily until it reaches a minimum value, indicating that the model has converged to the optimal solution.

The learning rate (α) is an important hyperparameter that determines the size of each update step. Its value significantly affects the convergence speed of the algorithm.

- **Small Learning Rate (e.g., $\alpha = 0.001$):** The algorithm converges slowly because the parameter updates are very small. Although it reaches the minimum accurately, it requires many iterations.
- **Moderate Learning Rate (e.g., $\alpha = 0.01$):** The algorithm converges efficiently and reaches the minimum cost in fewer iterations. This provides a good balance between speed and stability.
- **Large Learning Rate (e.g., $\alpha = 0.5$):** The algorithm takes very large steps, which may overshoot the minimum point. As a result, the cost may oscillate or even increase, preventing convergence.

Gradient Descent has a variation called Stochastic Gradient Descent (SGD). Unlike Gradient Descent, which updates the parameters using the entire training dataset, SGD updates the parameters after processing one training sample at a time. This makes SGD faster and more suitable for large datasets, although its updates are noisier and may fluctuate before converging

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Stochastic Gradient Descent (SGD) is an optimization algorithm used to train machine learning models by minimizing the cost function. Unlike Batch Gradient Descent (BGD), which updates the model parameters after processing the entire training dataset, SGD updates the parameters after each individual training sample. This makes SGD computationally faster and more suitable for large datasets, although its updates are noisier.

The cost function for a regression problem is the Mean Squared Error (MSE):

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- $J(\theta)$ = Cost function
- y_i = Actual output
- $\hat{y}_i$ = Predicted output
- n = Number of training samples

The parameter update rule for Batch Gradient Descent is:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial J}{\partial \theta}$$

where the gradient is computed using all training samples before updating the parameters.

In Stochastic Gradient Descent, the parameters are updated after processing each training example:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial J_i}{\partial \theta}$$

where J_i is the loss for a single training sample and α is the learning rate.

In this experiment, SGD is implemented for a simple regression problem and its convergence is compared with Batch Gradient Descent. Initially, both algorithms start with randomly initialized parameters. Batch Gradient Descent computes the gradient using the complete dataset, resulting in smooth and stable convergence but requiring more computation per iteration. In contrast, SGD updates the parameters after every sample, allowing it to reach a good solution much faster, although the loss fluctuates due to frequent updates. After several iterations, both algorithms converge to similar optimal values, but SGD usually reaches the solution in less time for large datasets

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Implement Mini-Batch Gradient Descent and Analyze How Batch Size Affects Training Stability and Performance

Mini-Batch Gradient Descent is an optimization algorithm that combines the advantages of Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD). Instead of updating the model parameters using the entire dataset (Batch GD) or a single training sample (SGD), Mini-Batch Gradient Descent updates the parameters after processing a small group (batch) of training samples. This approach improves computational efficiency while providing more stable convergence than SGD.

The cost function for a regression problem is the Mean Squared Error (MSE):

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where:

- $J(\theta)$ = Cost function
- y_i = Actual output
- $\hat{y}_i$ = Predicted output
- n = Number of training samples

The model parameters are updated using the Gradient Descent update rule:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \frac{\partial J}{\partial \theta}$$

where:

- θ = Model parameters
- α = Learning rate
- $\frac{\partial J}{\partial \theta}$ = Gradient computed from each mini-batch

In this experiment, the dataset is divided into small batches (for example, batch sizes of 16, 32, or 64 samples). The model processes one batch at a time, computes the gradient, updates the parameters, and repeats the process until all batches are completed. This procedure is repeated for multiple epochs until the cost function converges.

The batch size plays a significant role in determining the training speed and stability. A small batch size (e.g., 16) updates the weights more frequently, resulting in faster learning but with larger fluctuations in the loss. A medium batch size (e.g., 32) provides a good balance between convergence speed and stability. A large batch size (e.g., 64 or higher) produces smoother convergence because the gradients are averaged over more samples, but each update requires more computation and memory.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Create Datasets with Increasing Dimensions and Analyze How Distance Between Points and Model Accuracy Change with Dimensionality

The Curse of Dimensionality refers to the problems that arise when the number of features (dimensions) in a dataset increases. As dimensionality grows, the data points become increasingly sparse, making it difficult for machine learning algorithms to identify meaningful patterns. Distance-based algorithms such as K-Nearest Neighbors (KNN) and clustering methods are particularly affected because the distances between data points become very similar in high-dimensional spaces.

The distance between two points is commonly calculated using the Euclidean Distance:

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

where:

- x_i and y_i = Coordinates of two data points
- n = Number of dimensions
- d = Euclidean distance

In this experiment, datasets are created with increasing numbers of dimensions such as 2, 10, 50, and 100 features. For each dataset, the average distance between data points and the performance of a machine learning model are analyzed. As the number of dimensions increases, the average distance between points also increases, while the difference between the nearest and farthest neighbors becomes very small. This makes it difficult for the model to distinguish between similar and dissimilar data points.

The increase in dimensionality also affects model accuracy. With a small number of dimensions, the model learns meaningful patterns and achieves high accuracy. As the number of dimensions increases, the dataset becomes sparse, requiring more training data to achieve the same level of performance. If sufficient data is not available, the model may suffer from overfitting or reduced prediction accuracy